

Revisão AOC I

ACH2055 - Organização e Arquitetura de Computadores II

Valdinei Freire da Silva

Escola de Artes, Ciências e Humanidades - USP

2025

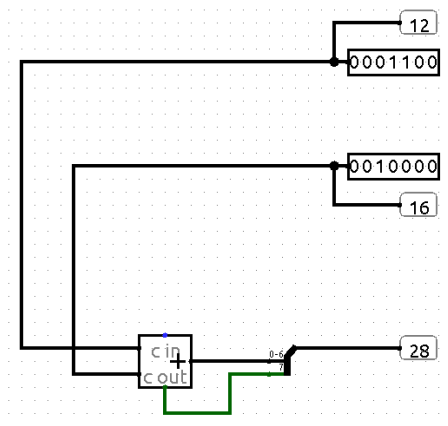
- ▶ O que vocês precisam saber?
- ▶ Sistemas de Numeração
 - ▶ notação científica
 - ▶ prefixos (nano, pico, mega, giga, etc.)
 - ▶ conversões: binário, decimal, hexadecimal
- ▶ Circuitos Digitais
 - ▶ expressão lógica
 - ▶ tabela verdade
 - ▶ circuito lógico
- ▶ Organização
 - ▶ Memória
 - ▶ ULA (Unidade Lógica e Aritmética)
 - ▶ Unidade de Controle
 - ▶ Multiplexadores
 - ▶ Registradores

Problema

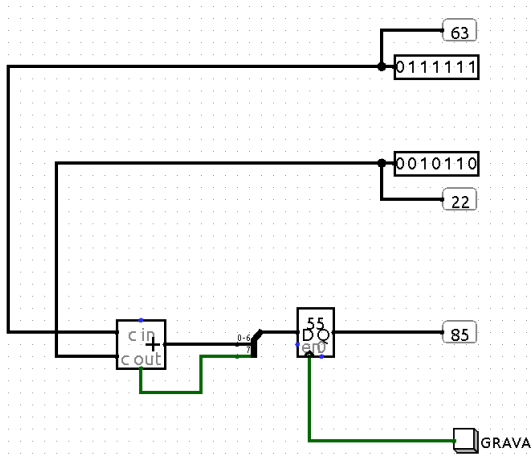
Problema Construa uma máquina M (circuitos digitais utilizando blocos + conjunto de instruções + programa) que interaja com outras N máquinas $M_1, M_2, \dots, M_N$ por meio de registradores especiais da seguinte maneira:

- ▶ Para cada máquina M_i existem dois registradores: um de entrada e um de saída
- ▶ As palavras nos registradores especiais possuem 8 bits com o seguinte significado: (i) o bit d_7 indica se existe dado novo para a máquina M ler ($d_7 = 1$) ou se o dado já foi lido ($d_7 = 0$) e (ii) os bits restantes $d_6, \dots, d_0$ indicam um número inteiro binário
- ▶ Quando aparecer um novo dado da máquina M_i , a máquina M deve ler esse dado e somar com os dados anteriores correspondentes à mesma máquina M_i
- ▶ O resultado da soma para cada máquina M_i deve ser colocado no registrador de saída correspondente

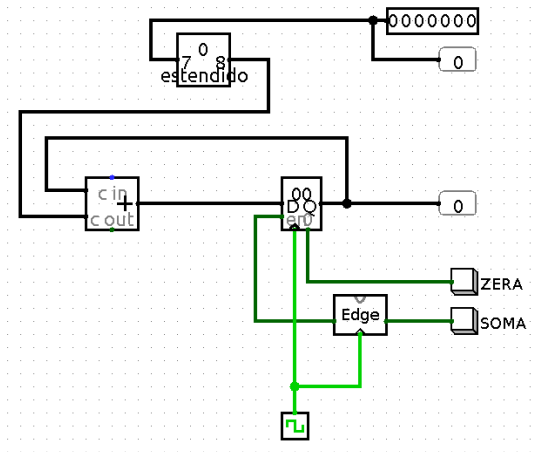
Circuito Combinacional



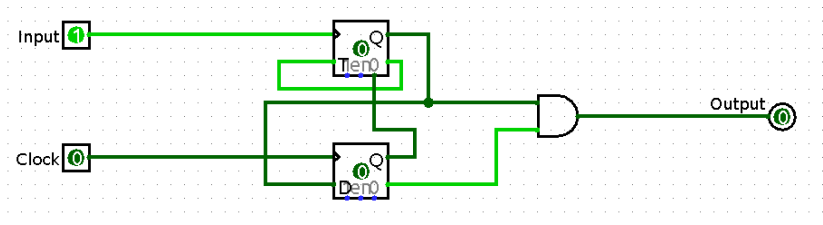
Circuito Sequencial - Registrador de Saída



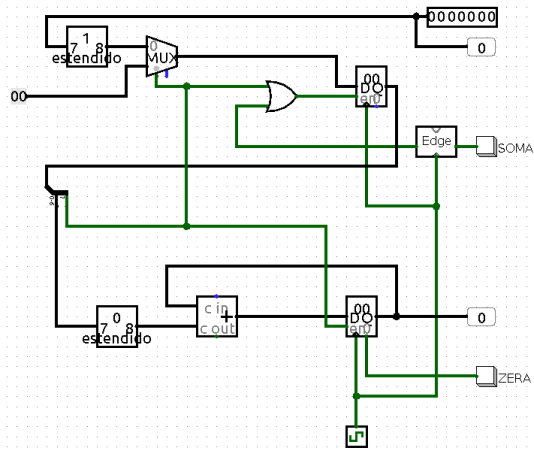
Circuito Sequencial - Acumulador



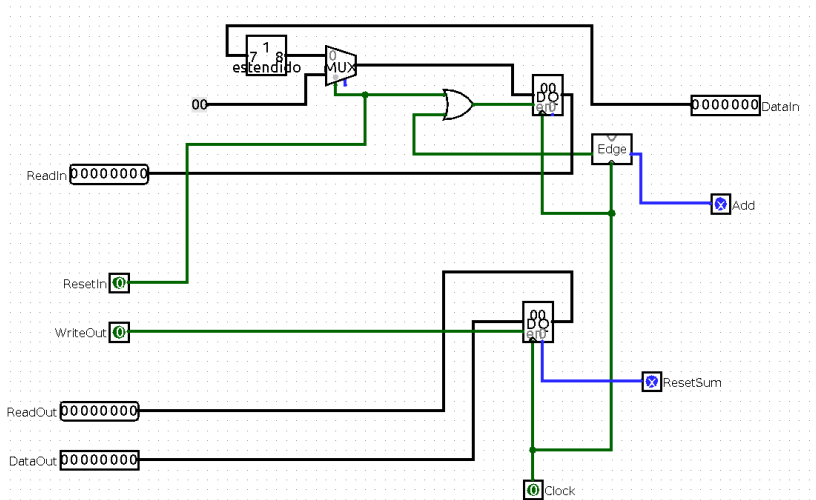
Detector de Subida



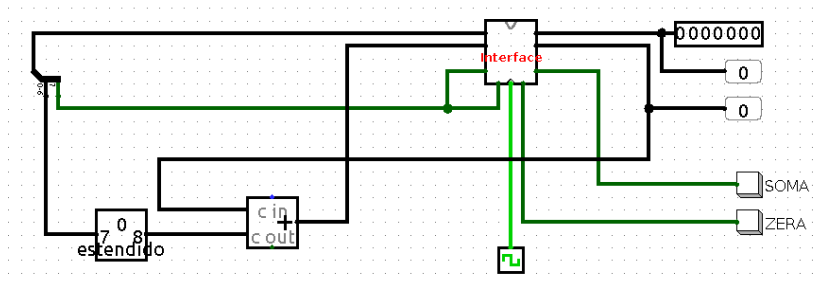
Circuito Sequencial - Registrador de Entrada



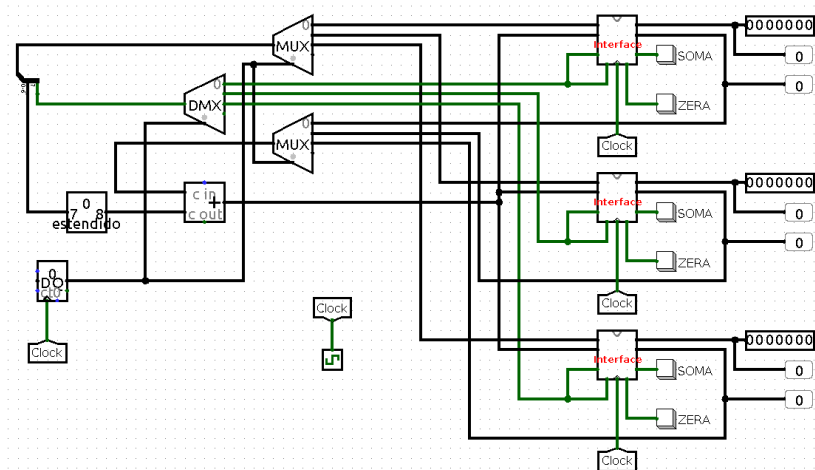
Interface de Entrada e Saída



Circuito Sequencial - Registrador de Entrada



Arbitrando várias Interfaces



Conjunto de Instruções

- ▶ `add_acc Ix` - considerando a Interface `Ix`, se o bit mais significativo (bit 7) de `ReadIn` for 1, soma o conteúdo dos bits restantes (bit 6 ao 0) com o valor em `ReadOut` e coloca o resultado em `DataOut` e habilita `ResetIn` e `WriteOut`, caso contrário não faz nada.
- ▶ `jump A` - a próxima ação a ser executada encontra-se no endereço `A` da ROM.

Codificação das Instruções

Considerando que a memória ROM armazena palavras de 11 bits e possuem endereçamento de 10 bits:

- ▶ `add_acc Ix` - código da operação (0) + filler (8 bits) + código da interface (2 bits)
- ▶ `jump A` - código da operação (1) + endereço da ROM (10 bits)

Programa

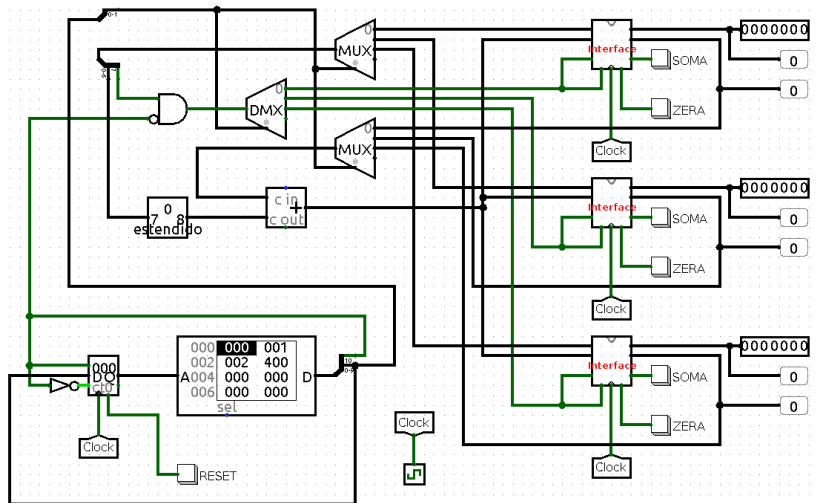
Considerando que $N = 3$ e a tabela abaixo de códigos:

Interface	Código
I1	000000
I2	000001
I3	000010

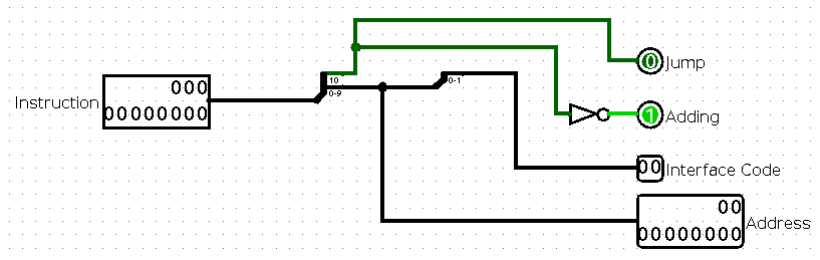
Programa:

Assembly	Cod. Binário	Cod. Hexadecimal
add_acc I0	0 00000000 00	000
add_acc I1	0 00000000 01	001
add_acc I2	0 00000000 10	002
jump 0	1 0000000000	400

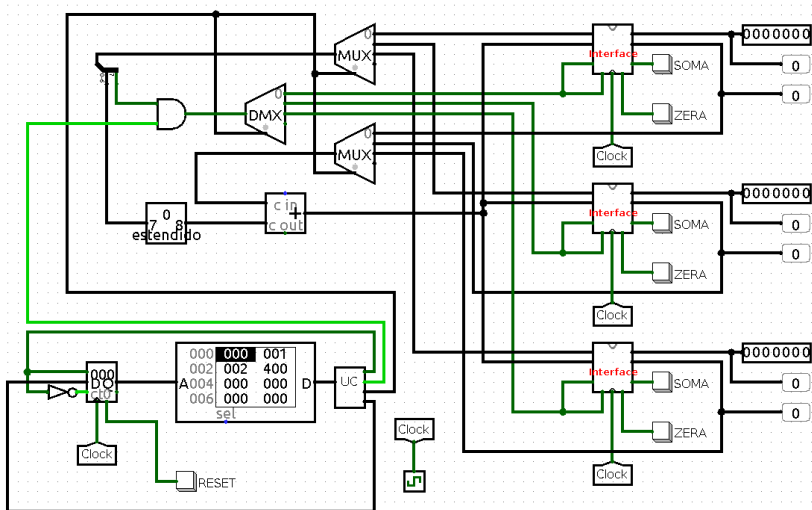
Máquina M



Unidade de Controle



Máquina M

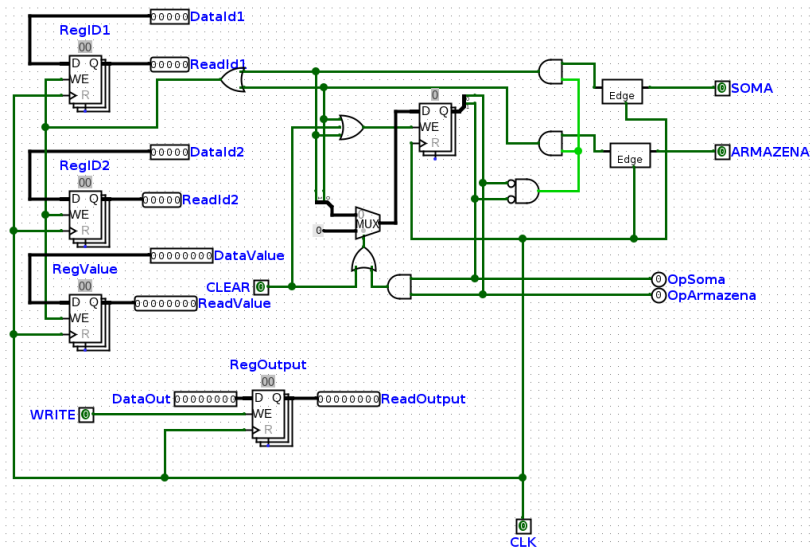


Problema 2 Construa uma máquina M que preste serviço de armazenamento e soma para outra máquina cliente por meio de uma interface de entrada e saída:

- ▶ a máquina cliente pode fornecer um identificador Id e um valor $Value$ que devem ser armazenados
- ▶ a máquina cliente pode requisitar a soma entre os valores de dois identificadores arbitrários

Problema 3 Construa duas máquinas $M1$ e $M2$ que calcule o MDC (Máximo Divisor Comum) entre dois números em dois cenários: com e sem circuito de divisão.

Problema 2



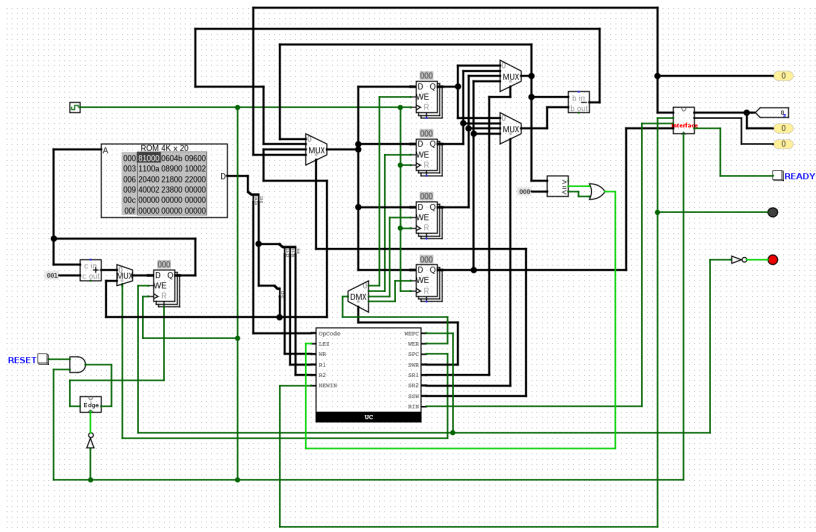
Problema 3 com Divisão

Label	Assembly	Cod. Binário	Cod. Hex
LOOP:	lri \$1, 255	000001 01 000000001111	050ff
	lri \$2, 75	000001 10 00000000110	0604b
	mod \$0, \$1, \$2	000010 00 01 10 00000000	08600
	blez \$0, END	000100 00 00000000111	10007
	mv \$1, \$2	001000 01 10 0000000000	21800
	mv \$2, \$0	001000 10 00 0000000000	22000
	jump LOOP	010000 00000000000010	40002
END:	mv \$3, \$2	001000 11 10 0000000000	23800
	halt	000000 00000000000000	00000

Programa 3 sem Divisão

Label	Assembly	Cod. Binário	Cod. Hex
LOOP:	lri \$1, 255	000001 01 000000001111	050ff
	lri \$2, 75	000001 10 000000000110	0604b
	sub \$1, \$1, \$2	000010 01 01 10 00000000	09600
	blez \$1, END	000100 01 000000001010	1100a
	sub \$0, \$2, \$1	000010 00 10 01 00000000	08900
	blez \$0, LOOP	000100 00 000000000010	10002
	mv \$0, \$1	001000 00 01 0000000000	20400
	mv \$1, \$2	001000 01 10 0000000000	21800
	mv \$2, \$0	001000 10 00 0000000000	22000
	jump LOOP	010000 000000000000010	40002
END:	mv \$3, \$2	001000 11 10 0000000000	23800
	halt	000000 0000000000000000	00000

Problema 3



EDVAC - Electronic Discrete Variable Automatic Computer

LOAD	x	$AC \leftarrow M[x]$
STORE	x	$M[x] \leftarrow AC$
ADD	x	$AC \leftarrow AC + M[x]$
SUB	x	$AC \leftarrow AC - M[x]$
SHIFT LEFT		$AC \leftarrow AC \times 2$
SHIFT RIGHT		$AC \leftarrow AC/2$
JUMP	x	$PC \leftarrow x$
JGE	x	if $AC \geq 0$ then $PC \leftarrow x$
LOAD ADR	x	$AC \leftarrow M[x].address_field$
STORE ADR	x	$M[x].address_field \leftarrow AC$

One Instruction Set Computer

SUBLEQ a, b, c $M[b] \leftarrow M[b] - M[a]$
 if $M[b] \leq 0$ then $PC \leftarrow c$

```
.macro    add        a b  
          subleq    a z  
          subleq    z b  
          subleq    z z  
  
.macro    copy       src dst  
          subleq    dst dst  
          add        src dst
```